

Echelon Embed Architecture

Stopniowe podmienianie ekranów starej aplikacji na Echelon

dealer-fx-app · zespół

2026-05-06

Echelon — Architektura embed/micro-frontend dla migracji legacy Angular

Dokument opisuje **konkretną architekturę** osadzania widoków Echelon (Angular 21) w istniejącej aplikacji Angular w starszej wersji (AngularJS 1.x, Angular 2-12, lub innej). Celem jest **podmiana kawałkami** zamiast big-bang rewrite.

Bazuje na rzeczywistym pakiecie `@echelon-framework/embed` z `host-bridge` przez `postMessage`.

1. Założenia i ograniczenia

- **Stara aplikacja:** Angular X.x (znana wersja, ale różna niż 21). Może używać NgModule, RxJS 6, własnego routera, własnego DI.
 - **Echelon:** Angular 21, standalone components, signals, OnPush.
 - **Heterogenous environment:** dwie wersje Angulara w jednym DOM = ryzyko (multiple zone.js, NgZone collisions, RxJS conflicts).
 - **Backend:** wspólny — Echelon konsumuje te same API co legacy.
 - **Auth:** pojedyncze SSO/token; legacy jest source of truth.
 - **Cel migracji:** 12-24 miesiące, ekrany podmieniane fragmentami, zero downtime.
-

2. Wybór technologii integracji

Trzy podejścia. Każde ma trade-offy.

2.1. iframe + postMessage (REKOMENDOWANE)

- **Izolacja:** pełna — dwie wersje Angulara nie kolidują (różne window, różne zone.js)

- **Bezpieczeństwo:** same-origin lub cross-origin z CSP, X-Frame-Options: SAMEORIGIN
- **Komunikacja:** tylko przez postMessage (typed contract)
- **Stylowanie:** theme propagowany przez init message; layout fixed (iframe size)
- **Routing:** lokalny w iframe, globalny w hoście; deep-link przez query params
- **Deploy:** embed = osobna aplikacja Echelon, deploy niezależny
- **Wady:** iframe size management (auto-resize), modale “obcięte” do granic iframe (delegacja do hosta)

2.2. Angular Elements (Web Components)

- **Bez iframe:** Echelon component eksportowany jako <echelon-dashboard> custom element
- **Bardziej zintegrowany layout:** brak iframe granicy
- **Risk:** dwie wersje Angulara w tym samym window — możliwe **zone.js conflicts**, NgZone collisions
- **Bundle size:** każdy element = pełny Angular runtime (kilkaset KB)
- **Auth/theme:** przez properties + DOM events
- **Wady:** niestabilność z różnymi wersjami Angulara, wymaga Zone.js noop lub ngZone: 'noop'

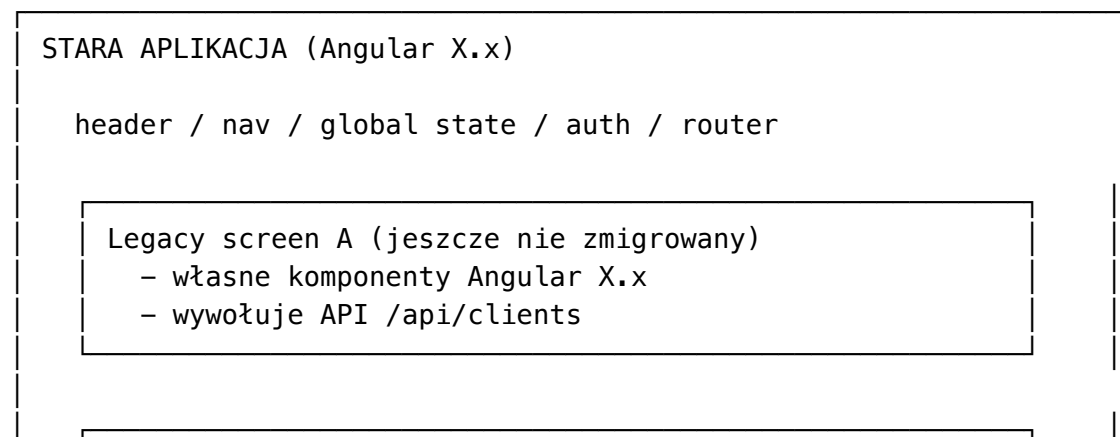
2.3. Module Federation (Webpack 5)

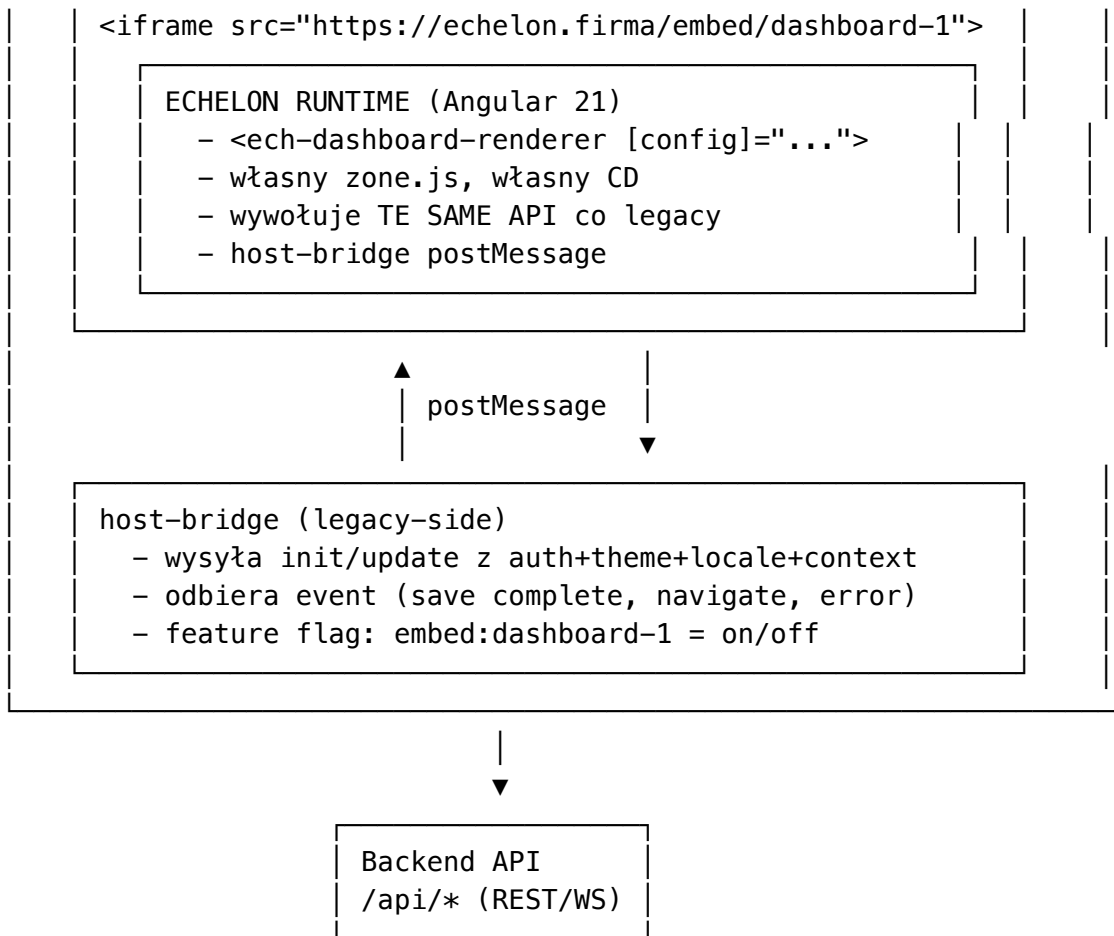
- **Współdzielenie kodu:** jedna instancja Angulara, hosting modułów zdalnych
- **Wymaga:** identyczna lub kompatybilna wersja Angulara po obu stronach
- **Niemożliwe** dla legacy AngularJS / Angular 2-12 + Echelon (Angular 21) — niekompatybilne
- **Sens tylko:** gdy migrujesz **jednocześnie** legacy do Angular 21

Decyzja

Idziemy w iframe + postMessage: jedyne podejście dające pełną izolację dwóch różnych wersji Angulara, predictable, wspierane przez @echelon-framework/embed natywnie.

3. Architektura wysokopoziomowa





Kluczowe punkty:

- **Stary host pozostaje shellem** — zarządza header, nav, globalnym stanem, autoryzacją, routerem
- **Każdy zmigrowany ekran** to iframe ładowany z URL Echelon app
- **Host-bridge** to cienka warstwa po stronie legacy (~3KB JS) która komunikuje się z iframe
- **Backend nie zmienia się** — jeden endpoint, dwa konsumenci (legacy i Echelon)

4. Setup po stronie Echelon app (embed-side)

Echelon app to **osobna aplikacja Angular 21** zbudowana z `@echelon-framework/*`. Jej job: - Wystawiać dashboards/formularze/listy jako self-contained widoki dostępne pod `/embed/<componentId>` - Reagować na `postMessage` `init/update` od hosta - Emitować `postMessage` event (`save`, `navigate request`, `error`)

4.1. Bootstrap

```
// echelon-app/src/main.ts
import { bootstrapApplication } from '@angular/platform-browser';
import { provideEchelon } from '@echelon-framework/runtime';
import { initEmbedHostBridge } from '@echelon-framework/embed/host-bridge';
```

```
bootstrapApplication(EmbedShellComponent, {
  providers: [
    provideEchelon({
      widgets: [...allWidgets],
      validators: { ... },
      formatters: { ... },
    }),
  ],
}).then(() => {
  initEmbedHostBridge({
    allowedOrigins: ['https://legacy.firma.com'], // restrict cross-
      origin
    onInit: (componentId, data) => {
      // data zawiera { sessionId, userId, role, theme, locale,
      // params }
      sessionStorage.set(data);
    },
    onUpdate: (componentId, data) => {
      sessionStorage.update(data);
    },
  });
});
```

4.2. Routing — /embed/<componentId>

```
// echelon-app/src/routes.ts
export const routes: Routes = [
  { path: 'embed/:componentId', component: EmbedRendererComponent },
];
```

Komponent EmbedRendererComponent ładuje PageConfig po componentId (z bundle albo z GET /api/embed/contract/<componentId>) i renderuje przez <ech-dashboard-renderer>.

4.3. Wysyłanie eventów do hosta

```
import { sendEventToParent } from '@echelon-framework/embed/host-
  bridge';

// Po zapisie formularza:
sendEventToParent('save-complete', { entityId: 'CLI-001' });

// Prośba o nawigację globalną:
sendEventToParent('navigate-request', { route: '/clients/CLI-001' });

// Błąd:
sendEventToParent('error', { code: 'PERM_DENIED', message: '...' });
```

5. Setup po stronie hosta (legacy app)

5.1. Włączenie skryptu embed

W index.html legacy aplikacji:

```
<script src="https://echelon.firma.com/embed.js"></script>
<script>
  EchelonEmbed.init({
    host: 'https://echelon.firma.com',
    allowedOrigins: ['https://echelon.firma.com'],
  });
</script>
```

5.2. Tag deklaracyjny (najprostsze)

W szablonie legacy:

```
<echelon-embed
  component="dashboard-clients-1"
  data-context='{ "clientId": "CLI-001", "tenantId": "acme" }'
></echelon-embed>
```

5.3. API imperatywne (gdzie dane są dynamiczne)

```
// Legacy controller / component
const handle = EchelonEmbed.mount('#dashboard-container', {
  componentId: 'dashboard-clients-1',
  data: {
    sessionToken: this.authService.getToken(),
    userId: this.user.id,
    role: this.user.role,
    theme: this.theme.current(), // np. 'dark-default'
    locale: this.i18n.current(), // 'pl-PL'
    params: { clientId: 'CLI-001' },
  },
  onEvent: (event) => {
    if (event.type === 'save-complete') {
      this.refreshList();
    }
    if (event.type === 'navigate-request') {
      this.$router.navigate(event.payload.route);
    }
    if (event.type === 'error') {
      this.toast.error(event.payload.message);
    }
  },
});
```

// Aktualizacja sesji w trakcie (np. po refresh tokena):

```
handle.update({ sessionId: newToken });
```

```
// Unmount (przy nawigacji):
handle.destroy();
```

5.4. Feature flag

Każdy zmigrowany ekran ma flagę. Stara wersja zostaje jako fallback.

```
if (this.featureFlags.isOn('embed:dashboard-clients-1')) {
  return this.renderEchelonEmbed();
} else {
  return this.renderLegacyDashboard();
}
```

6. Kontrakt komunikatów (TypeScript types)

Wspólny dla obu stron. Można umieścić w @firma/shared-types lub repo per ekran.

```
/** Wszystkie komunikaty mają prefix 'echelon:'. */
export type EchelonMessage =
  // Host → Embed
  | { type: 'echelon:init'; componentId: string; data: HostContextData }
  | { type: 'echelon:update'; componentId: string; data:
    Partial<HostContextData> }
  | { type: 'echelon:command'; componentId: string; command:
    HostCommand }
  // Embed → Host
  | { type: 'echelon:ready'; componentId: string }
  | { type: 'echelon:event'; componentId: string; event: string;
    payload: unknown }
  | { type: 'echelon:resize'; componentId: string; height: number }
  | { type: 'echelon:contract'; componentId: string; contract:
    ComponentContract }
  | { type: 'echelon:error'; componentId: string; code: string;
    message: string };

export interface HostContextData {
  readonly sessionId: string;
  readonly userId: string;
  readonly role: string;
  readonly theme: string;
  readonly locale: string;
  readonly params: Record<string, unknown>;
  readonly tenantId?: string;
  readonly featureFlags?: Record<string, boolean>;
}

export type HostCommand =
```

```
| { kind: 'reload' }  
| { kind: 'focus'; fieldId: string }  
| { kind: 'submit' }  
| { kind: 'cancel' }  
| { kind: 'reset' };
```

Zasada: nieznan typ komunikatu → ignorowany (forward-compat). Dodawanie nowych typów bez breaking change.

7. Auth, session, theme, locale

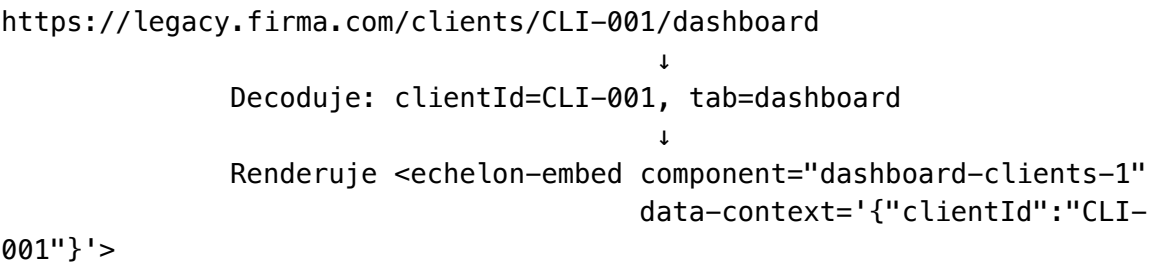
Element	Source of truth	Propagacja
Auth token	legacy host (z SSO/auth service)	init.data.sessionToken na start, update.data.sessionToken przy refresh
User identity	legacy host	init.data.userId/role/tenantId
Theme	legacy host	init.data.theme (id z BUILT_IN_THEMES Echelon); embed wywołuje applyTheme()
Locale	legacy host	init.data.locale; embed konfiguruje DraftTranslationStore
Feature flags	legacy host	init.data.featureFlags (subset relevant for embed)

Embed **nie czyta** localStorage hosta — single source of truth = postMessage.

Logout: legacy wysyła command: { kind: 'reload' } lub po prostu iframe.src = ''.

8. Routing i deep linking

8.1. URL legacy hosta zachowuje stan ekranu



8.2. Embed routing wewnętrzny

W iframe, Echelon Angular Router obsługuje wewnętrzne tabs/wizard steps. Niezależny od hosta.

8.3. Globalna nawigacja

Embed nie nawiguje hosta bezpośrednio. Wysyła event: `navigate-request`:

```
sendEventToParent('navigate-request', { route: '/clients/CLI-002' });
```

Host decyduje czy honoruje (np. odpalić inny embed, albo legacy screen).

9. Datasource — wspólny backend

Echelon DS wskazują na **te same endpointy** co legacy:

```
// echelon-app dashboard config
const config: PageConfig = {
  page: {
    id: 'dashboard-clients-1',
    datasources: {
      clients: {
        kind: 'transport',
        transport: 'http',
        endpoint: '/api/clients/${session.tenantId}', // dynamiczny endpoint
        headers: { Authorization: 'Bearer ${session.sessionToken}' },
      },
    },
    // ...
  },
};
```

`$session.*` to wartości z `init.data` przekazane przez host. DS jest wstrzykiwany z tym kontekstem.

Zaleta: zmiana backendu (np. nowy URL API) nie wymaga zmian w embed — config jest deklaratywny.

10. Risk control

10.1. Feature flag per embed

Każdy embed = osobna flaga. Off → legacy fallback. On → Echelon.

```
const flagKey = `embed:${componentId}`;
if (featureFlags.isOn(flagKey)) renderEmbed(); else renderLegacy();
```


10.2. Wersjonowanie bundle

Echelon app deploy: każda wersja 0.7.0, 0.7.1 ma własny URL /embed/v0.7.1/...
Stara wersja dostępna do natychmiastowego rollback.

```
EchelonEmbed.init({ host: 'https://echelon.firma.com/v0.7.1' });
```

10.3. Canary rollout

Procent użytkowników (np. 5%) dostaje embed. Pozostali legacy. Stopniowo zwiększaj %.

```
const percent = featureFlags.percent('embed:dashboard-1'); // 0..100
const myBucket = hash(userId) % 100;
if (myBucket < percent) renderEmbed(); else renderLegacy();
```

10.4. Telemetry

Host loguje: - Czas mount-to-ready embed (init → ready event) - Liczba error events z embed - Liczba event events (signal “user is doing things”) - Network timing (per /api/embed/contract + /api/clients)

Wykres w Grafana / Datadog. Threshold alarmów: error rate > 1%, time-to-ready > 3s.

10.5. Plan rollback

1. Feature flag → off (instant rollback do legacy)

2. Embed bundle → poprzednia wersja (/v0.7.0/)

3. Backend nie zmieniany — bez ryzyka po stronie API

11. Pułapki (z doświadczenia)

#	Pułapka	Mitigacja
1	CSP frame-src w ho- ście blokuje iframe Echelon	Dodaj https://echelon.firma.com do frame-src w meta CSP lub headers
2	X-Frame-Options Eche- lon serwer ustawia na DENY	Zmień na SAMEORIGIN lub usuń (CSP frame-ancestors zamiast)
3	Third-party cookies (cross-origin iframe) blo- kowane przez Chrome/Safari	Hosting Echelon w subdome- nie legacy (echelon.firma.com jako dziecko firma.com); session token przez postMessage nie cookies
4	CORS — Echelon ifra- me pobiera	Backend musi zwracać Access-Control-Allow-

#	Pułapka	Mitigacja
	/api/clients, host blokuje	Origin: https://echelon.firma.com lub origin matchuje host (same-domain)
5	zone.js patches double gdy embed Angular ładuje swoje, host ma swoje	Iframe = OSOBNY window = osobne zone.js. Brak konfliktu. (Inaczej w Angular Elements!)
6	localStorage shared między iframe a hostem (same-origin) — niebezpieczne dla auth	Embed NIE pisze do localStorage tokens — używa tylko in-memory state z postMessage
7	Layout broken — iframe ma fixed height, content w środku jest wyższy	echelon:resize event → host aktualizuje iframe.style.height
8	Modale “obcięte” do granic iframe	Embed wysyła event: open-modal → host renderuje modal NA POZIOMIE host (nie w iframe)
9	Drag & drop poza iframe nie działa cross-frame	Wewnątrz iframe OK; cross-frame DnD wymaga shared protocol (rzadko potrzebne)
10	Browser back button w iframe nie nawiguje hosta	Embed używa history.replaceState + emituje event: navigate-request zamiast pushState
11	Auth refresh token wygasł — embed pokazuje 401	Host nasłuchuje event: auth-expired, refreshuje token, wysyła update.sessionToken
12	i18n niesynchronizowane — host ma pl-PL, embed ma en-US	Theme + locale ZAWSZE z init.data, nie z embed defaults

12. Plan migracji — 4 fazy

Faza 1 — Przygotowanie (1-2 sprinty)

- ☐ Backend: weryfikacja CORS + auth middleware dla nowego origin
- ☐ Hosting Echelon app: subdomena echelon.firma.com, deploy pipeline (TC PublishGA)
- ☐ Echelon app skeleton: bootstrapApplication, provideEchelon, initEmbedHostBridge
- ☐ Host-side: embed.js w legacy, kontener <echelon-embed> lub helper EchelonEmbed.mount
- ☐ Kontrakt komunikatów: typed in @firma/shared-types
- ☐ Feature flag system (jeśli brak)

- ☐ Wybór 1 niskoryzykowny ekran read-only jako PoC (np. dashboard analytics)

Faza 2 — Pierwszy embed end-to-end (1 sprint)

- ☐ PageConfig dla wybranego ekranu (designer Echelon)
- ☐ Iframe w legacy podmienia stary komponent
- ☐ init/update/event flow działa
- ☐ Theme + locale zsynchronizowane
- ☐ Feature flag on/off przełączna w runtime
- ☐ E2E test: parity functional z legacy (te same dane, te same akcje)

Faza 3 — Skalowanie (n sprintów)

- ☐ Tabela migracyjna: kolumny screen, complexity, status, flag, pct, last-error-rate
- ☐ Priorytety: read-only → CRUD → BPMN flows
- ☐ Każdy ekran: PageConfig w designerze → bundle commit → deploy → feature flag canary 5% → 50% → 100%
- ☐ Stara wersja kodu **usuwana** dopiero po 100% i 2 tygodniach bez rollbacku
- ☐ Telemetria każdego embed monitorowana

Faza 4 — Inwersja (cel końcowy)

- ☐ Wszystkie istotne ekrany w Echelon
- ☐ Stara apka = tylko shell (auth + nav + globalny state)
- ☐ Refactor shellu na natywny <ech-app-shell> lub minimalna wrapperka
- ☐ Wycofanie starej aplikacji

13. Wskaźniki sukcesu

Per embed:

- **Time to ready** (init → ready event) $\leq 1.5s$ p95
- **Error rate** (error events / sessions) $\leq 0.5\%$
- **Save success rate** $\geq 99\%$
- **Browser performance**: Lighthouse score > 80 (mobile)
- **A11y**: bez regresji WCAG AA z legacy

Globalnie:

- **% ekranów zmigrowanych** (target po 12 miesiącach: 60-80%)
- **Średni czas migracji 1 ekranu** ≤ 1 sprint
- **Wzrost satysfakcji users** (NPS, ankieta)
- **Spadek bugfix-time** (nowy stack łatwiejszy do debug)

14. Kiedy zatrzymać migrację embed

- Aplikacja małej (< 5 ekranów) → big-bang rewrite jest tańszy
 - Backend wymaga jednoczesnej zmiany → najpierw migracja backendu
 - Aplikacja będzie wyłączona w < 12 miesięcy → koszt embed-infra się nie zwróci
 - Brak dostępu do legacy hostu (vendor) → tylko drop-in replacement, nie embed
-

15. Checklisty

Setup hosta (legacy)

- ☐ CSP zawiera `frame-src https://echelon.firma.com`
- ☐ `<script src="https://echelon.firma.com/embed.js">` załadowane
- ☐ `EchelonEmbed.init()` wywołane raz przy bootstrap
- ☐ `Helper mountEchelonEmbed(slot, componentId, ctx)` wprowadzony do legacy DI
- ☐ Feature flag system gotowy
- ☐ Telemetry forward + Grafana dashboards

Setup embed (Echelon app)

- ☐ `bootstrapApplication` z `provideEchelon`
- ☐ `initEmbedHostBridge({ allowedOrigins, onInit, onUpdate })`
- ☐ Routing `/embed/:componentId` → `renderer`
- ☐ `PageConfig` per dashboard (z designera lub bundle)
- ☐ Auth interceptor: wstrzykuje `sessionToken` z `$session` do każdego HTTP requesta
- ☐ Theme + locale stosowane z `init.data`
- ☐ Auto-resize: na mount + DOM mutation observer → `sendEventToParent('resize', {height})`

Per-screen migracja

- ☐ `PageConfig` zwalidowany przez Zod
 - ☐ Designer: theme parity z legacy
 - ☐ E2E test obejmuje all critical paths
 - ☐ Feature flag stworzony
 - ☐ Rollback procedure udokumentowana
 - ☐ Owner zespołu zatwierdza canary release
-

16. Pytania otwarte (do decyzji per projekt)

1. **Hosting:** subdomena (`echelon.firma.com`) vs path (`firma.com/echelon/`) — subdomena = czystsze CORS, path = brak third-party cookies issue
2. **Bundle distribution:** jeden Echelon app dla całej firmy czy osobny per legacy app (multi-tenant)?
3. **Versioning:** semver per bundle czy jeden global (v1/v2)?

4. **A11y**: czy framework Echelon jest zgodny z naszym a11y baseline (WCAG AA)?
 5. **Print/Export**: czy embed musi generować PDFy/CSVy w iframe, czy delegować do hosta?
-

— *koniec dokumentu*